

Bases de Dados - Aula 16

Projeto de Banco de Dados

Modelagens Conceitual e Lógica

Eduardo Corrêa Gonçalves

Sumário

<i>Projeto de Banco de Dados</i>	3
1. Etapas de um Projeto de BD	3
2. Projeto Conceitual	4
2.1 Entidades	4
2.2 Membros de Entidades	4
2.3 Relacionamentos.....	5
2.4 Cardinalidade de Relacionamentos	6
2.5 Atributos	9
2.6 Outras Técnicas de Modelagem	10
2.8 Um DER Completo	13
2.9 Notação Pé-de-Galinha (Crow's Foot).....	14
2.10 Exercícios sobre Modelagem Conceitual	15
3. Projeto Lógico	16
3.1 Mapeamento ER-Relacional.....	16
3.2 Chave Substituta.....	23
3.3 Notação Pé-de-Galinha em um Modelo Lógico	24
3. Desnormalização	25
4. Um Exemplo Passo a Passo	26
5. Exercícios	29

Projeto de Banco de Dados

Nesta aula será apresentada uma introdução concisa das técnicas que, juntamente com a teoria da normalização, são utilizadas para a **elaboração de projetos de banco de dados** de sistemas e aplicativos profissionais que utilizam **bancos de dados relacionais**.

1. Etapas de um Projeto de BD

A Figura 1 apresenta as etapas normalmente envolvidas na elaboração de um projeto de BD.

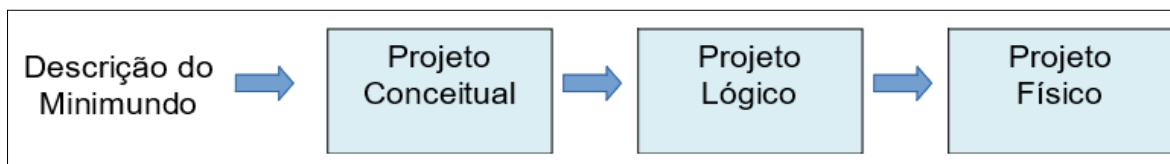


Figura 1. Diferentes etapas do processo de implementação de um banco de dados

A partir da descrição do minimundo a ser modelado, a primeira etapa consiste na elaboração do **projeto** (ou **modelo**) **conceitual**. Este modelo conterá uma descrição formal, porém em alto nível do BD. O modelo registrará “o que” irá aparecer no BD, sem fornecer detalhes sobre a forma “como” aparecerá. A técnica padrão para a elaboração de um projeto conceitual é a modelagem entidade-relacionamento, abordada na Seção 2. A modelagem conceitual é muito importante em qualquer projeto e possui um aspecto positivo muito interessante: ela pode ser aplicada a qualquer tipo de SGBD, seja ele relacional, orientado a documentos, orientado a grafos ou outro. Ou seja, o projeto conceitual independe do modelo de banco de dados.

A segunda etapa consiste na conversão do modelo conceitual para um **projeto** (ou **modelo**) **lógico**, já dependente do SGBD que será utilizado. Para SGBDs relacionais, o modelo consiste basicamente no conjunto de tabelas, campos, chaves e demais restrições de integridade que serão definidas no BD (ao final etapa, o script SQL DDL para a criação desses objetos estará produzido). É importante observar que as tabelas criadas deverão respeitar as formas normais apresentadas no capítulo anterior. Para mapear um modelo conceitual para um modelo lógico, utiliza-se um conjunto de regras e procedimentos padronizados. Além disso, esta fase costuma ser implementada com o auxílio de um software especialmente elaborado para este propósito. Este tipo de software é conhecido como **ferramenta CASE** (*Computer Aided Software Engineering*) ou simplesmente “**ferramenta para modelagem de dados**”. A Seção 3 desta aula será dedicada à modelagem lógica.

Por fim, a etapa de projeto físico cobre as especificações do BD em termos de estruturas de armazenamento, índices e implementação de técnicas direcionadas para o aumento do desempenho do BD. Este curso não apresenta detalhes sobre esta etapa, apresentando, no entanto, alguns conceitos básicos sobre índices e outros objetos de BD em uma de nossas aulas sobre tópicos extras.

2. Projeto Conceitual

O projeto conceitual tem por objetivo fornecer uma descrição em alto nível das principais estruturas de informação do BD, seja qual for o modelo de SGBD que você utilizará em seu projeto (relacional ou NoSQL). A técnica padrão para a elaboração de um projeto conceitual é a modelagem entidade-relacionamento (modelagem E/R), que é representada graficamente através de um diagrama entidade-relacionamento (DER). Esta seção apresenta os principais elementos e conceitos da modelagem E/R: entidade, relacionamento, atributo, generalização/especialização e auto-relacionamento.

2.1 Entidades

Define-se como entidade todo objeto existente no minimundo, que possua uma identificação distinta e um significado próprio. Ou, de uma maneira mais simples, uma entidade representa qualquer coisa (objeto) sobre o qual existe algum interesse em mantermos dados armazenados. Alguns exemplos:

- No BD sobre cinema, a principal entidade é *Filme*; outras entidades importantes são *Artista* e *País*.
- Em um BD contendo dados de uma pesquisa domiciliar do IBGE, as principais entidades normalmente são *Domicílio* e *Pessoa*. Outros exemplos entidades relevantes neste minimundo são *Unidade da Federação*, *Município* e *Localidade*.
- Em um BD utilizado por um site de comércio eletrônico, alguns exemplos de entidades são *Cliente*, *Produto*, *Pedido*, *Forma de Pagamento*, etc.

Em um DER, uma entidade é representada por um retângulo contendo seu nome (Figura 2).



Figura 2. Representação gráfica das entidades *Pessoa* e *Domicílio* no DER.

2.2 Membros de Entidades

Assim como uma relação é um conjunto de tuplas, uma entidade pode ser vista como um conjunto de elementos – ou membros – que possuem as mesmas propriedades. A Figura 3 apresenta um exemplo que ilustra este conceito, onde a entidade *Pessoa* é representada como um conjunto contendo três membros: ‘Thomas’, ‘Jane’ e ‘George’.

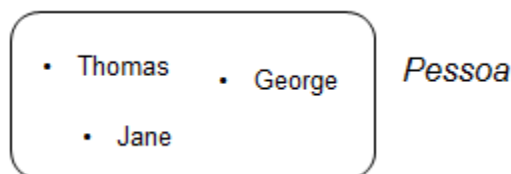


Figura 3. Membros da entidade *Pessoa*

2.3 Relacionamentos

Um relacionamento representa um fato ou acontecimento que liga duas entidades. Em um projeto conceitual, é comum (embora não obrigatório) expressar os relacionamentos através de verbos, enquanto as entidades são nomeadas com substantivos no singular. Observe o seguinte exemplo:

- *Pessoa mora em Domicílio.*

Neste exemplo, *Pessoa* e *Domicílio* representam entidades. O verbo “morar” liga estas duas entidades, ou seja, expressa o relacionamento entre elas. Outros exemplos:

- *Domicílio está localizado em alguma Unidade da Federação.*
- *Cliente realiza Pedido. Um Pedido é pago através de algum tipo de Forma de Pagamento (cartão de crédito, dinheiro, cheque, etc.).*

Em um DER, a forma mais comum de especificar um relacionamento é através de um losango ligado por linhas às entidades associadas, como mostrado na Figura 4. A indicação do nome do relacionamento é opcional, e pode ser feita tanto dentro como fora do losango.

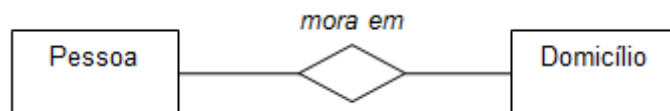


Figura 4. Representação gráfica do relacionamento “mora em” entre as entidades *Pessoa* e *Domicílio* no DER.

Uma forma interessante de compreender o relacionamento entre entidades é através da construção de um **diagrama de ocorrências**, como o apresentado na Figura 5.

Neste tipo de diagrama, as duas entidades envolvidas são representadas como conjuntos contendo alguns membros. As associações entre membros de duas entidades são representadas por linhas que ligam um membro de uma entidade com um membro da outra entidade. Estas associações descrevem com precisão as características do relacionamento entre as entidades (como comentaremos mais adiante). Observe ainda que o relacionamento propriamente dito também é representado como um conjunto no diagrama. Ele é posicionado entre as duas entidades e contém os pares de membros associados como seus elementos.

De acordo com a explicação apresentada, vamos examinar o diagrama de ocorrências da Figura 5. Este diagrama mostra que existe um relacionamento entre a pessoa p_1 e o domicílio d_1 . Isto indica que a pessoa p_1 reside em d_1 . De maneira análoga, p_2 e p_3 também estão relacionados com d_1 (as pessoas p_2 e p_3 moram em d_1) e p_4 está relacionado com d_2 (p_4 mora em d_2). Note ainda que d_3 não está relacionado com nenhum membro de *Pessoa*, indicando que o domicílio d_3 não possui nenhum morador.

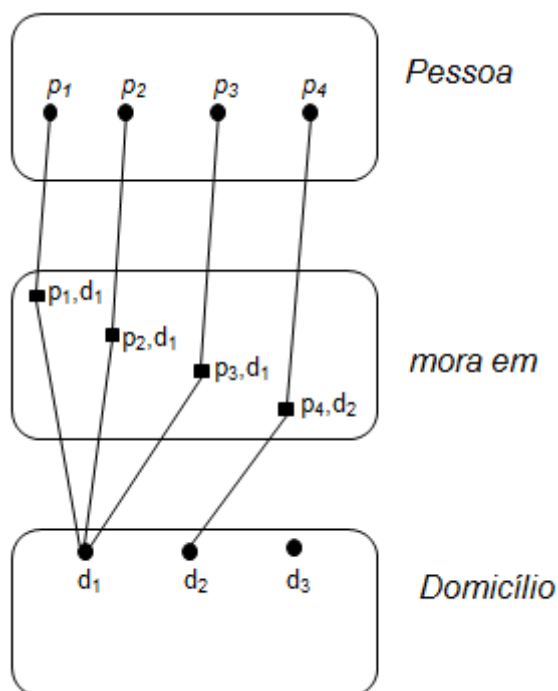


Figura 5. Diagrama de ocorrências do relacionamento “Pessoa mora em Domicílio”

2.4 Cardinalidade de Relacionamentos

A propriedade mais importante de um relacionamento é a sua **cardinalidade**. Considerando duas entidades A e B a cardinalidade de A pode ser definida como número mínimo e máximo de membros desta entidade que podem estar relacionados a um único membro de B .

Para que o conceito fique claro, considere novamente o relacionamento “Pessoa mora em Domicílio”. Conforme mostrado no diagrama de ocorrências da Figura 5, um membro de *Pessoa* pode estar associado a no mínimo um e no máximo um domicílio. Em outras palavras, uma pessoa reside em exatamente um domicílio. Por sua vez, um membro de *Domicílio* pode estar relacionado a no mínimo zero (como é o caso de d_3) e no máximo vários membros de *Pessoa* (como é o caso de d_1). Isto é, um domicílio pode ter de zero a muitos moradores residentes. Diz-se então que:

- *Pessoa* possui cardinalidades mínima e máxima iguais a 1 no relacionamento “mora em”.
- *Domicílio* possui cardinalidade mínima 0 e cardinalidade máxima N (muitos) no relacionamento “mora em”.

No DER, a cardinalidade é representada da forma apresentada na Figura 6. A anotação “0,N” ao lado da entidade *Pessoa* expressa que um membro do *Domicílio* (entidade do lado oposto) pode estar associado com nenhum ou muitos membros de *Pessoa*. Da mesma forma, a anotação “1,1” ao lado de *Domicílio* expressa que um membro de *Pessoa* (entidade do lado oposto) deve estar associado a um e somente um membro de *Domicílio*.

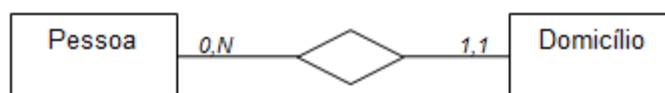


Figura 6. Representação gráfica das cardinalidades mínima e máxima do relacionamento “*Pessoa mora em Domicílio*” no DER

Observações importantes:

- A cardinalidade mínima pode ser 0 ou 1.
 - No primeiro caso, diz-se que a entidade possui **participação opcional** no relacionamento, isto é, alguns de seus membros podem não estar associados a nenhum membro da outra entidade. No exemplo apresentado, este é o caso da entidade *Domicílio*, uma vez que pode existir um domicílio sem morador.
 - No segundo caso, diz-se que a entidade possui **participação obrigatória** no relacionamento, pois cada um de seus membros precisa estar associado a algum membro da outra entidade. É o caso da entidade *Pessoa*, pois não pode existir uma pessoa sem domicílio associado.
- A cardinalidade máxima pode ser 1 ou N.
 - Na maioria das vezes, **apenas a cardinalidade máxima costuma ser indicada no DER**, pois ela é bem mais importante para fins de projeto de BD, conforme será evidenciado na Seção 1.3. Sendo assim, o relacionamento entre *Pessoa* e *Domicílio* é classificado como **1 x N**, podendo ser desenhado da forma mostrada na Figura 7.

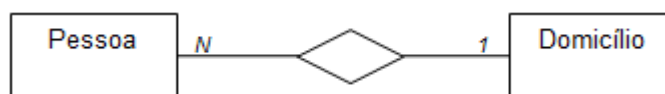


Figura 7. Representação do relacionamento “mora em” entre as entidades *Pessoa* e *Domicílio* no DER – apenas a cardinalidade máxima é indicada (forma mais comum)

Outros exemplos de relacionamentos 1 x N são apresentados na Figura 8. Estes exemplos apresentam entidades e relacionamentos tipicamente presentes em pesquisas sociais e econômicas do IBGE.

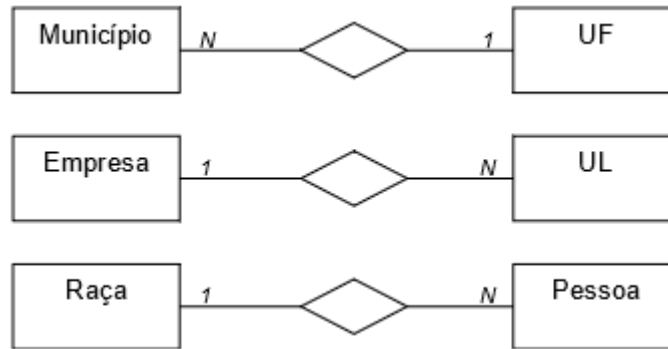


Figura 8. Exemplos de relacionamentos 1 x N

- *Município* e *UF*: Um município pertence a uma UF e uma UF têm muitos municípios.
- *Empresa* e *UL*: Uma empresa pode possuir muitas ULs (unidades locais) e uma UL pertence a apenas uma empresa.
- *Raça* e *Pessoa*: Uma pessoa entrevistada em uma pesquisa possui apenas uma raça ('Branca', 'Preta', 'Amarela', 'Parda' ou 'Índigena'). Uma raça pode estar associada a diversas pessoas.

Além de 1 x N, os relacionamentos também podem ser classificados em **1 x 1** e **N x N**. Em um relacionamento 1 x 1 entre as entidades *A* e *B*, um membro de *A* pode estar relacionado a no máximo um membro de *B* e vice-versa. Um exemplo é mostrado na Figura 9, em um minimundo onde assume-se que um presidente pode comandar apenas uma empresa e uma empresa possui um único presidente.

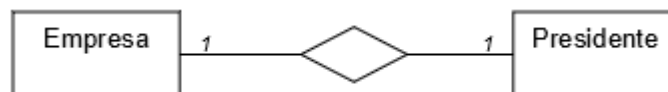


Figura 9. Exemplo de relacionamento 1 x 1

Finalmente, em um relacionamento N x N entre *A* e *B*, um membro de *A* pode estar relacionado a muitos um membros de *B* e vice-versa. Veja alguns exemplos na Figura 10.

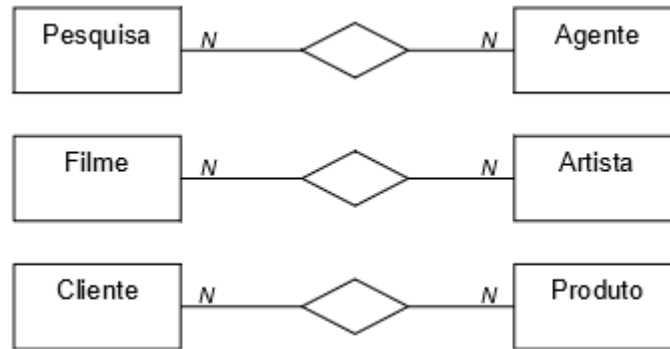


Figura 10. Exemplos de relacionamentos N x N

- *Pesquisa e Agente*: Uma pesquisa pode possuir diversos agentes (técnicos) de coleta. Um mesmo agente pode atuar em diversas pesquisas.
- *Filme e Artista*: O elenco de um filme possui muitos artistas. Um mesmo artista pode atuar em muitos filmes.
- *Cliente e Produto*: Um cliente pode comprar muitos produtos. Um produto pode ser comprado por vários clientes.

ATENÇÃO!

Alguns profissionais de BD consideram a notação para representação da cardinalidade um pouco confusa, pois em um relacionamento entre as entidades *A* e *B*, **a cardinalidade de *A* vai anotada do outro lado do relacionamento, ao lado de *B*.**

Por este motivo, existem algumas poucas ferramentas case que trocam o lado da anotação. Este é o caso, por exemplo, da ferramenta DBDesigner, um software livre muito utilizado para a elaboração de DER. Desta forma, é preciso ter atenção quando você utilizar alguma ferramenta e também quando você for ler artigos sobre projeto de BD na Internet.

Em resumo, a grande maioria das ferramentas e artigos adota o padrão introduzido na Figura 7: a cardinalidade de uma entidade é anotada do outro lado do relacionamento. Porém, existem algumas ferramentas que trocam o lado da anotação.

2.5 Atributos

O conceito de atributo de entidade é o mesmo de atributo de relação, ou seja, os atributos são propriedades que caracterizam os membros da entidade. No DER, atributos podem ser representados graficamente por uma oval contendo seu nome (existem outras notações, mas esta é a mais utilizada). Os atributos chave devem ser sublinhados. A Figura 11 apresenta a nossa conhecida entidade *Filme* e seus atributos.

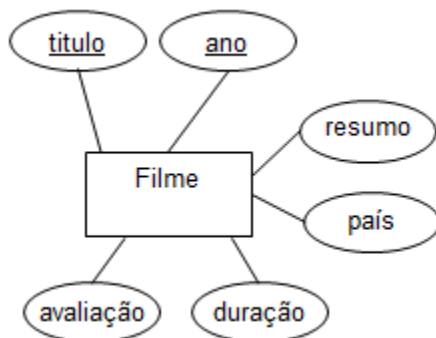


Figura 11. Entidade *Filme* e seus atributos

Relacionamentos também podem possuir atributos. Por exemplo, no relacionamento “*Cliente compra Produto*” (Figura 12), a quantidade comprada não é um atributo de *Cliente* (não caracteriza um cliente) e nem de *Produto* (não caracteriza um produto, pois estes são descritos por código e nome). Na realidade, a quantidade caracteriza uma compra específica de um produto por um cliente, consistindo assim em um atributo do relacionamento “compra”.

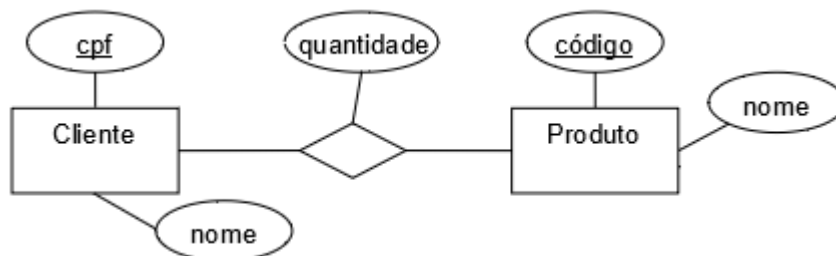


Figura 12. Atributo de relacionamento

Atributos no DER – uma Questão Prática

A notação para representar atributos adotada pelo DER costuma “poluir” o modelo. Por isso, é muito comum que profissionais elaborem o DER indicando apenas os principais atributos de cada entidade ou simplesmente sem desenhar nenhum atributo (apenas as entidades e relacionamentos são indicados).

2.6 Outras Técnicas de Modelagem

A seguir, são apresentadas algumas técnicas de modelagem avançadas que podem ser utilizadas por projetistas de BD.

Generalização e Especialização

Muitas vezes, uma entidade possui certos membros com propriedades especiais que não estão associadas aos demais. Por esta razão, pode ser útil definir entidades separadas para estes membros (entidades especializadas), com os seus atributos específicos.

Considere, por exemplo, um banco de dados de uma empresa que comercializa laptops e impressoras. Cada um desses tipos de produtos é identificado por seu “código” e possui um “fabricante” e um “preço”. No entanto, o produto laptop possui seus atributos específicos: “velocidade”, “tamanho da tela” e “tamanho do HD”. Da mesma forma, as impressoras também possuem atributos específicos: “tipo” (‘laser’ ou ‘jato de tinta’) e “colorida” (‘sim’ ou ‘não’). Em um DER, este tipo de situação pode ser modelada da forma apresentada na Figura 13.

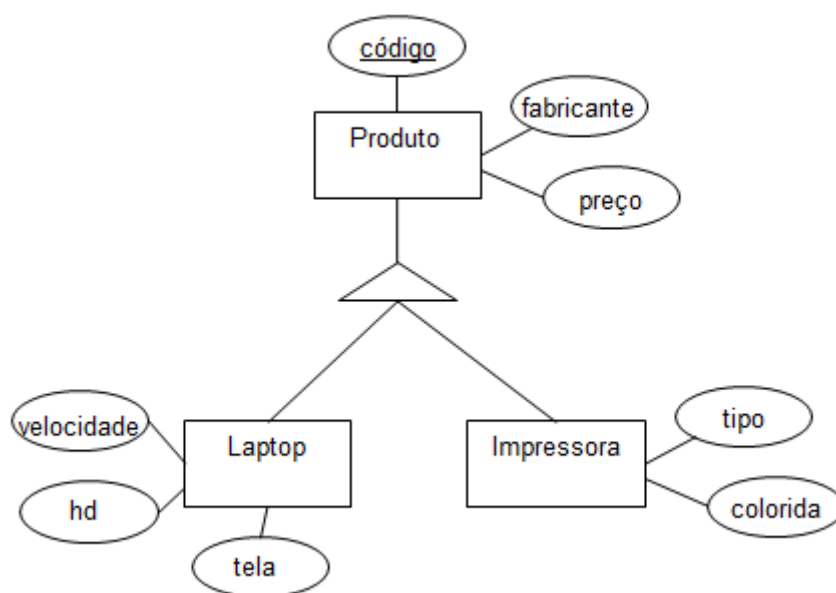


Figura 13. Generalização / especialização

Nesta representação, nós conectamos a entidade genérica (*Produto*) com as entidades especializadas (*Laptop* e *Impressora*) através de uma relação do tipo “é um” (“um *A* é um *B*”). No exemplo: “Um *Laptop* é um *Produto*” e “Um *PC* é um *Produto*”. Observe que, no DER, o símbolo para representar a generalização / especialização é um triângulo isósceles.

É importante observar que no relacionamento “é um”, as entidades especializadas herdam todas as propriedades da entidade genérica. Isto quer dizer que um laptop é descrito não apenas por sua velocidade, tamanho da tela e tamanho do HD, mas também por seu código, fabricante e preço. **O identificador de *Laptop* e *Impressora* é herdado da entidade genérica**, ou seja, é o “código” que identifica unicamente cada membro de *Laptop* e cada membro de *Impressora*.

Auto-relacionamento

Uma entidade auto-relacionada é aquela cujos membros podem se relacionar com outros membros que pertencem à própria entidade. A Figura 14 mostra um exemplo, onde a entidade *Filme* é auto-relacionada para que seja possível registrar as sequências de uma produção. Neste caso, torna-se possível modelar a seguinte situação: um filme possui diversas sequências, mas para cada sequência há apenas um filme original.

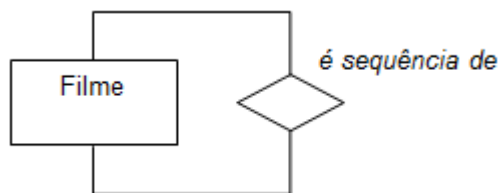


Figura 14. Auto-relacionamento

Caso o conceito esteja confuso, observe o exemplo de diagrama de ocorrências mostrado na Figura 15. Neste diagrama, foram acrescentadas setas nas linhas que representam as associações entre membros para que o “sentido” da associação ficasse explicitado (ex.: ‘O Império Contra-Ataca’ é sequência de ‘Guerra nas Estrelas’). O exemplo evidencia que a modelagem com o auto-relacionamento possibilita com que de maneira direta, a própria especificação da entidade *Filme* seja capaz de armazenar filmes e ainda representar todas as suas sequências.

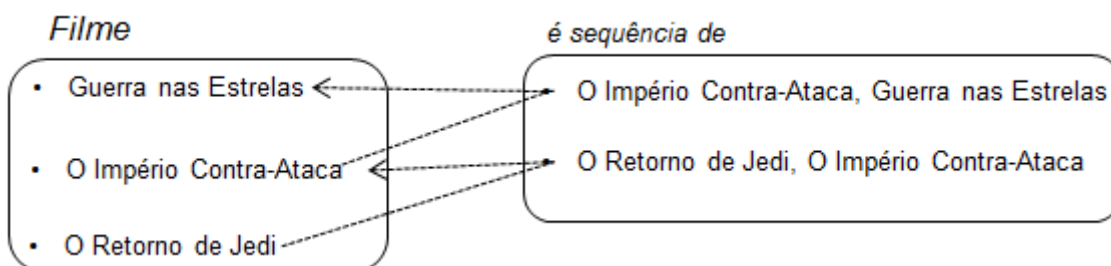


Figura 15. Diagrama de ocorrências da entidade auto-relacionada *Filme*

Relacionamentos Unários e Binários

Um auto-relacionamento é também chamado de relacionamento **unário**, uma vez que representa uma associação entre membros de uma mesma entidade.

Já os relacionamentos envolvendo duas entidades (ex: *Domicílio* e *Pessoa*, *Filme* e *Artista*, *Presidente* e *Empresa*, etc.), são chamados de **binários**, pois envolvem duas entidades. Estes são, definitivamente, os relacionamentos mais comuns na prática.

Também existem os relacionamentos envolvem três ou mais entidades. Estes tipos de relacionamento são menos comuns e mais complexos de serem modelados, não sendo cobertos em nossa disciplina.

2.7 Papéis

O papel de uma entidade em um relacionamento descreve a função cumprida por um membro da entidade no relacionamento em questão.

Embora a descrição de um papel possa ser utilizada em qualquer tipo de relacionamento, ela se torna especialmente útil em auto-relacionamentos. No exemplo da Figura 15, em cada associação um membro exerce o papel de ‘filme original’, enquanto o outro exerce o papel de ‘sequência’. A descrição deste papel torna clara o aspecto do minimundo representado na modelagem.

2.8 Um DER Completo

Para finalizar a seção, a Figura 16 um exemplo de DER completo, referente ao minimundo do site de cinema introduzido desde as nossas aulas iniciais (a versão definitiva foi criada na Aula 06). Não existe uma forma padrão de dispor os símbolos, mas uma boa dica para tornar o diagrama mais legível é tentar ao máximo evitar o cruzamento de linhas.

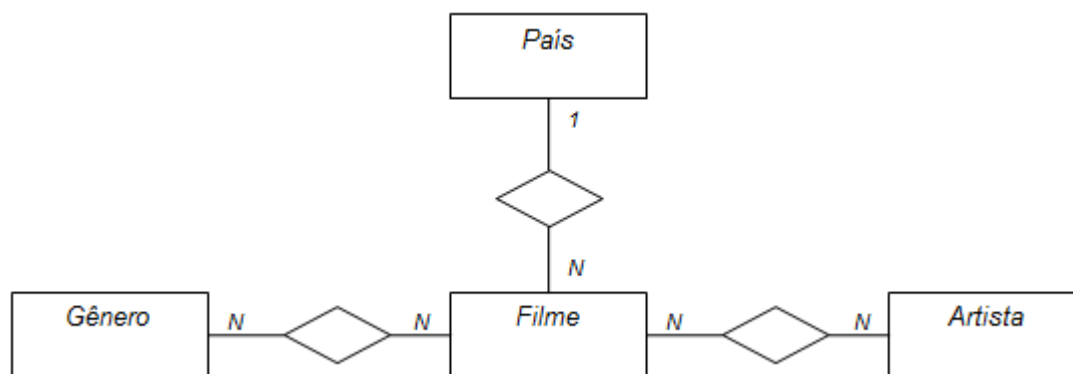


Figura 16. DER do site sobre cinema

Veja que o DER expressa que:

- Existem quatro entidades associadas a este problema: *Filme*, *País*, *Gênero* e *Artista*.
- Neste minimundo, considera-se que o relacionamento entre *Filme* e *País* é do tipo 1 x N. Ou seja, um filme é produzido em um único país¹ e um país pode produzir muitos filmes.
- Já os relacionamentos entre *Filme* e *Gênero* e *Filme* e *Artista* são N x N. Um filme pode possuir muitos gêneros (ex: ‘Drama’, ‘Romance’, ‘Aventura’) e um gênero pode estar associado a muitos filmes. Um filme normalmente irá possuir muitos artistas em seu elenco e um mesmo artista pode atuar em vários filmes.

Mas espere aí! Em diversas aulas anteriores mostramos que este mesmo BD com seis tabelas e dissemos que este número de tabelas corresponderia a um “bom projeto”:

Filme (titulo, ano, resumo, pais, duração, avaliação)

País (sigla, nome)

Artista (nome, sexo)

Gênero (nome)

FilmeGênero (titulofilme, ano, nomegenero)

FilmeElenco (titulofilme, ano, nomeartista)

¹ Na realidade sabemos que existem coproduções. Mas suponha que o site sobre filmes adote como critério registrar apenas o “principal” país responsável pela produção do filme.

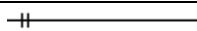
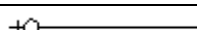
No entanto, agora estamos apresentando um DER com quatro entidades. Como isso é possível? Fique calmo, pois quando o DER da Figura 16 for convertido para um projeto lógico, isto resultará em um total de 6 tabelas a serem criadas no BD. Sendo assim, o DER realmente reflete o esquema em nossa base de dados de filmes! Na próxima aula você entenderá como este mapeamento é feito.

2.9 Notação Pé-de-Galinha (Crow's Foot)

A notação originalmente criada para representar um relacionamento consiste no uso de um losango, conforme mostrado na seção anterior. Essa notação foi criada por um pesquisador chamado Peter Chen, sendo por isso chamada de Notação de Chen. Porém, existem outras notações muito populares. Uma delas é a notação Pé-de-Galinha (Crow's Foot), que é extremamente prática e será brevemente introduzida nesta seção (utilizamos como base o texto do artigo apresentado em <https://www.vertabelo.com/blog/crow-s-foot-notation/>).

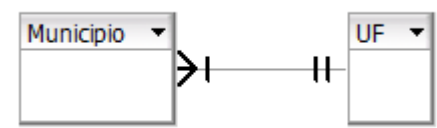
Assim como na notação de Chen, a notação Crow's Foot, também representa associações entre entidades através de linhas que ligam as entidades. Entretanto, as cardinalidades são expressas através de símbolos utilizados nos dois extremos das linhas, que são capazes de representar cardinalidades mínimas e máximas de um relacionamento. Estes símbolos são apresentados na Tabela 1.

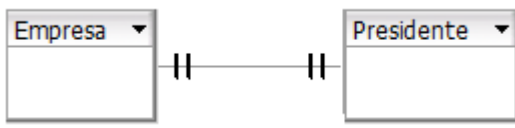
Tabela 1 – Símbolos utilizados pela Notação Crow's Foot

Símbolo	Descrição
	Zero ou mais
	Um ou mais
	Um e somente um
	Zero ou um

A Tabela 2, mostra diversos exemplos de relacionamentos representados na notação Crow's Foot.

Tabela 2 – Exemplos de Relacionamentos Expressos na Notação Crow's Foot

Exemplo	Descrição
	Relacionamento Nx1, com um lado opcional. Uma pessoa deve residir em exatamente um domicílio (um membro de pessoa está associado a exatamente um membro de domicílio). E um domicílio pode ter 0 ou mais pessoas (um membro de domicílio pode estar associado com 0 ou mais membros de Pessoa).
	Um município pertence a 1 UF e uma UF possui 1 ou mais municípios (nenhuma UF pode ter 0 municípios).

Exemplo	Descrição
	Relacionamento 1x1. Uma empresa possui obrigatoriamente 1 presidente e um presidente deve estar associado a exatamente 1 empresa.
	Neste caso, trata-se de um relacionamento NxN. Da maneira que está representado, o relacionamento expressa que um filme deve ter 1 ou mais atores (ou seja, é obrigatório ao menos um ator). E que um ator pode ter atuado em 0 ou mais filmes (posso ter algum membro de ator que não esteja relacionado a nenhum de filme).

Obs.: na aula sobre Projeto Lógico, veremos que uma forma fácil de memorizar a notação Crow's Foot é nunca esquecer que o pé-de-galinha estará sempre localizado ao lado da tabela que possui a chave estrangeira.

2.10 Exercícios sobre Modelagem Conceitual

(1) Um sistema que gerencia os **projetos de iniciação científica** de uma universidade possui as seguintes características:

- Cada projeto é identificado unicamente por um código, possui um título, ano, um orçamento previsto em R\$ e está associado a um único assunto (ex: Estatística, Física, Banco de Dados, etc.)
- Um assunto possui um nome e nenhum outro atributo.
- Um projeto possui um único pesquisador responsável e diversos alunos participantes.
- Pesquisadores possuem CPF, nome e nível (J=Júnior, P=pleno ou S=senior).
- Alunos possuem uma matrícula, nome, sexo e idade.

A partir do minimundo descrito, **elabore o projeto conceitual de um BD** para ser utilizado por este sistema.

(2) Lojas de um shopping vendem produtos a preços e quantidades variadas seus clientes. Esse minimundo poderia ser modelado através das relações abaixo:

- *Loja*(idLoja, nome)
- *Cliente*(idCliente, nome, endereço)
- *Produto*(idProduto, nome)
- *Compra*(idCompra, idCliente, idLoja, dataCompra)
- *ProdutoCompra*(idCompra, idProduto, quantidade, preço)

Desenhe um diagrama E/R que descreva este banco de dados.

(3) Considere um sistema que será criado para apresentar os resultados das Olimpíadas de Tóquio. Através desse sistema, os usuários poderão fazer consultas por país, modalidade e atleta. Abaixo alguns exemplos de consulta possíveis:

- Quais os nomes de todos os atletas que ganharam medalha de ouro?
- Listar os resultados completos de uma dada modalidade (ex.: “Vôlei de Praia Feminino” ou “Natação Livre 50m Masculino”)
- Quais países não inscreveram atletas do sexo feminino na Olimpíada?
- Quais as siglas e nomes de países que ganharam ao menos uma medalha de ouro na Olimpíada?

Considerando as consultas acima, **elabore o projeto conceitual de um BD** para ser utilizado por este sistema.

3. Projeto Lógico

O projeto lógico consiste na conversão do modelo conceitual para um esquema relacional de um determinado SGBD (SQLite, Oracle, MySQL, etc.). Este processo é chamado de **Mapeamento ER-Relacional** (*ER-to-Relational Mapping*). O objetivo final do projeto lógico é a obtenção de um script DDL para a geração de um conjunto de tabelas, campos e restrições de integridade em um SGBD relacional específico.

O projeto lógico pode ser elaborado através de uma receita padrão, que consiste na utilização de um conjunto de regras de mapeamento. A próxima subseção descreve as regras mais importantes

3.1 Mapeamento ER-Relacional

A seguir apresentam-se as principais regras de mapeamento um passo-a-passo simplificado para converter um DER em um projeto lógico.

REGRA 1 –Mapeamento de Entidades

O primeiro passo é realizar o mapeamento das entidades através da aplicação de uma regra muito simples: **toda entidade transforma-se numa tabela na base de dados.**

Tome como exemplo o DER do site sobre cinema, apresentado na Figura 2. Este diagrama possui quatro entidades: *País*, *Gênero*, *Filme* e *Artista*. Seguindo a regra recém-apresentada, cada uma deverá ser mapeada para uma tabela específica no BD (Figura 17).

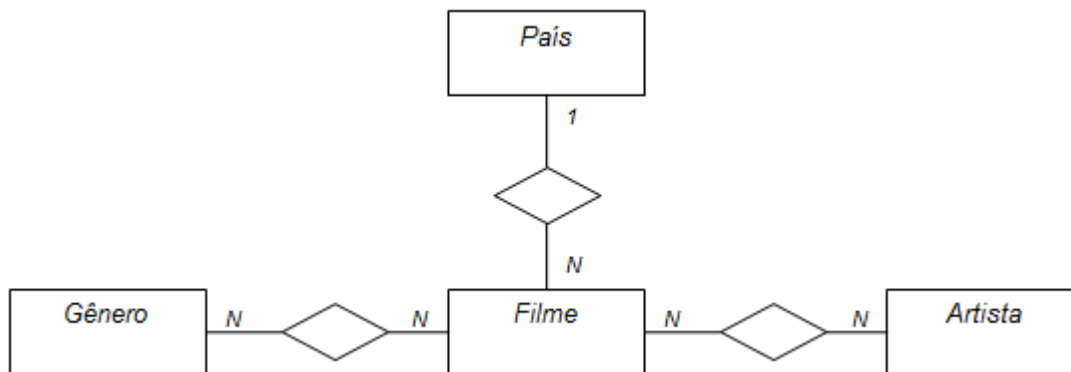


Figura 2. DER do site sobre cinema

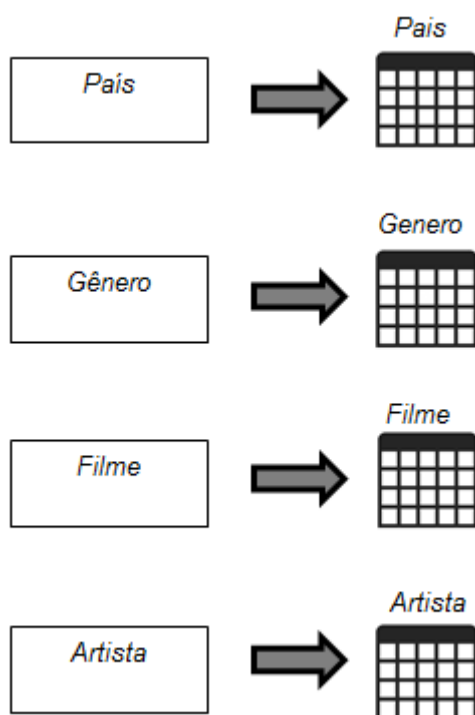


Figura 17. Regra 1 (exemplo no BD de filmes) - toda entidade do modelo conceitual se transforma em uma tabela do modelo lógico

Com relação aos atributos das tabelas, é importante considerar que no nível conceitual não precisamos nos preocupar muito com os seus tipos ou dar um tratamento elementar para essa questão (ex.: utilizar apenas “alfanumérico”, “inteiro” e “real”). Entretanto, quando estamos no nível lógico já é preciso definir os tipos de uma forma mais adequada, de acordo com as opções oferecidas pelo SGBD. A Tabela 1 apresenta exemplos de tipos de dados encontrados na maioria dos SGBDs relacionais, como Oracle, SQL Server, PostgreSQL, MySQL, etc.

Tabela 1 – Principais tipos de dados disponíveis nos SGBDs

Tipo de Dado	Descrição / Aplicação
CHAR(n), CHARACTER(n)	Texto de caracteres com tamanho fixo igual a “n”. Utilizado para colunas cujo tamanho é conhecido e fixo, como “sexo” (1 caractere), “cep” (8 caracteres), “cnpj” (14 caracteres), etc.
VARCHAR(n)	Texto com tamanho variável. O espaço que não for utilizado não ocupa espaço no banco de dados. Este campo é utilizado para colunas cujo tamanho é muito variável, como “endereço”, “nome” e “razão social”.
INT, INTEGER	Tipos numéricos utilizados para representar números inteiros.
NUM, REAL, FLOAT	Tipos numéricos utilizados para representar números reais.
NUMBER	No SGBD Oracle, pode ser utilizado para armazenar números inteiros ou reais.
BOOLEAN	Tipo booleano, onde um valor 0 é considerado FALSE e 1 é considerado TRUE.
DATE	Para armazenar uma data (dia/mês/ano).
DATETIME	Para armazenar data e hora com precisão de segundos (ou milissegundos, dependendo do SGBD).
TIMESTAMP, TIME	Para armazenar data e hora com precisão de milissegundos (ou superior, dependendo do SGBD).
CLOB, TEXT	Pode armazenar textos volumosos (ex.: com até 4GB).
BLOB	Pode armazenar arquivos diversos (JPG, AVI, XLS, DOC, etc).

Como exemplo, a Figura 18 mostra um possível mapeamento da entidade *País* do modelo conceitual para a tabela *País* do modelo lógico. O mapeamento inclui a especificação do nome da tabela e de todos os seus atributos. Para cada atributo, especifica-se o nome, tipo, se corresponde à chave primária e se é um campo NOT NULL. Sendo assim, veja que a tabela *País* possui dois campos: “sigla”, do tipo CHAR(2), chave primária e NOT NULL; e “nome” do tipo VARCHAR(30), NOT NULL.

Tabela País			
Nome da Coluna	Tipo	PK?	NOT NULL
sigla	CHAR(2)	X	X
nome	VARCHAR(30)		X

Figura 18. Definição da tabela País

Regra 2 –Mapeamento de Relacionamentos 1 x N

Esta classe de relacionamento é a mais comum no mundo real. Conforme vimos anteriormente, em uma relação 1 x N entre as entidades *A* e *B*, um membro de *A* pode relacionar-se com muitos membros da entidade *B*, mas cada membro de *B* somente pode estar relacionado a um membro da entidade *A*.

No DER de filmes, um exemplo deste tipo de relacionamento é o que ocorre entre *País* e *Filme*: um país pode produzir muitos filmes e um filme é produzido em um único

país. Outro exemplo é o relacionamento entre *Domicílio* e *Pessoa*, apresentado na última aula (um domicílio pode ter muitas pessoas e uma pessoa mora em um único domicílio).

A regra de mapeamento conceitual-lógico para relacionamentos 1 x N é a seguinte:

1. Identificar a relação *A* cujos membros podem se relacionar com muitos membros de *B*.
2. Incluir como chave estrangeira em *B*, a chave primária de *A*.

A Figura 19 apresenta esta situação na base de dados de filmes: a chave primária da tabela *Pais* teve que ser transportada como chave estrangeira para a tabela *Filme*, para que o relacionamento entre estas duas tabelas pudesse ser implementado. Observe que na tabela *Pais*, o nome do campo que armazena a sigla do país é “sigla”, enquanto na tabela *Filme* o nome deste campo é “pais”².

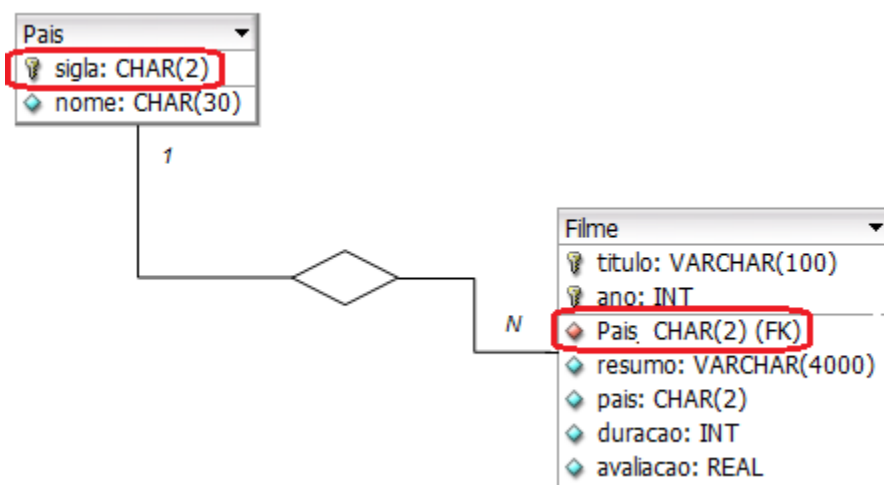


Figura 19. Regra 2 (exemplo no BD de filmes) - o campo “sigla” (chave primária de *Pais*) também está presente em *Filme* (com o nome de “pais”) com o objetivo de implementar o relacionamento 1 x N entre as entidades *Pais* e *Filme*

IMPORTANTE: perceba que **não faria sentido** “fazer o inverso”, isto é, jogar a chave primária de *Filme* (“titulo” e “ano”) para a tabela *Pais*. Se você fizer isso, só conseguirá armazenar um filme para cada linha de país (ou seja, o BD só suportaria um filme por país!!!).

² Conforme sabemos, não importa que os nomes dos campos sejam diferentes nas tabelas, o que vale é definir uma restrição de FOREIGN KEY no comando CREATE TABLE para que seja possível implementar uma chave estrangeira.

Regra 3 – Realizar o Mapeamento de Relacionamentos 1 x 1

Esta classe de relacionamento ocorre com muito menos frequência na prática. Em uma relação 1 x 1 entre as entidades *A* e *B*, cada membro de *A* pode relacionar-se com no máximo um membro de *B*, e vice-versa.

No DER de filmes, não existe nenhum relacionamento 1 x 1. Porém, na Figura 20 apresentamos o exemplo “*Empresa* possui *Presidente*”, onde assumimos que um presidente pode comandar apenas uma empresa e uma empresa possui um único presidente.

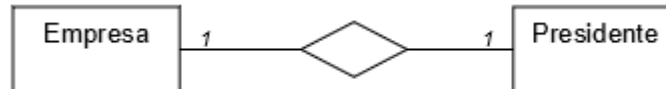


Figura 20. Exemplo de relacionamento 1 x 1

A abordagem mais comum para realizar este tipo de mapeamento é implementar uma chave estrangeira. Mas neste caso, diferentemente do que ocorre no mapeamento 1 x N, **qualquer uma das tabelas poderá receber a chave estrangeira**. Na Figura 21, apresenta-se um exemplo onde a PK de *Empresa* foi exportada como FK para *Presidente*. Como cada linha de *Presidente* corresponde a um presidente de uma empresa específica, o código da empresa é suficiente para representar de qual é empresa em questão.

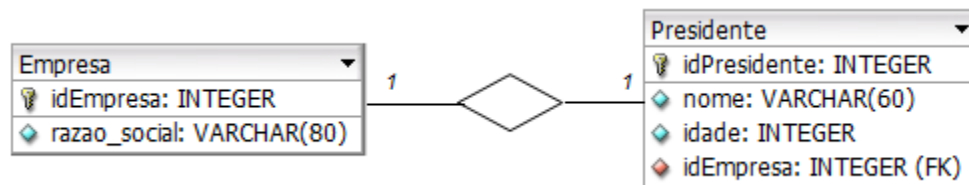


Figura 21. Regra 3 (exemplo) - o campo “*idEmpresa*” (chave primária de *Empresa*) também está presente em *Presidente* com o objetivo de implementar o relacionamento 1 x 1 entre as entidades *Empresa* e *Presidente*

Conforme mostrado acima, no mapeamento de relacionamentos 1 x 1, a FK pode ser implementada em qualquer uma das tabelas, já que isso não ocasionará qualquer problema relacionado ao armazenamento de dados no BD (diferentemente do que ocorre nos relacionamentos 1 x N, onde existe uma tabela certa para receber a FK).

No entanto, muitas vezes pode ser interessante observar a cardinalidade mínima para determinar a “melhor tabela” para receber a FK. Considere por exemplo, o relacionamento que representa a coordenação de um departamento em uma dada empresa, onde cada departamento pode ser chefiado por apenas um funcionário:

- *Funcionário* chefia *Departamento*.

Neste relacionamento, um departamento é chefiado por no mínimo 1 e no máximo 1 funcionário; e um funcionário pode chefiar no mínimo 0 e no máximo 1 departamento. Se

considerarmos uma empresa com cem funcionários e cinco departamentos, teremos uma situação onde noventa e cinco funcionários não chefiavam nenhum departamento! Em uma situação como esta, a melhor opção seria colocar a FK na tabela de departamentos (ou seja, transportar o “id do funcionário” como FK em departamento). Se fizermos o oposto (transportar o “id do departamento” como FK em funcionário) ficaremos com uma situação onde 95% dos registros ficarão com valor NULL para o “id do departamento”.

Regra 4 – Realizar o Mapeamento de Relacionamentos N x N

Neste grau de relacionamento, cada membro da entidade *A* pode relacionar-se com muitos membros da entidade *B* e vice-versa. Um exemplo é o relacionamento entre *Filme* e *Artista*: um filme pode ter muitos artistas. E um artista pode atuar em vários filmes.

Para este tipo de relacionamento, a seguinte regra deve ser aplicada: **todo relacionamento N x N transforma-se em uma tabela no projeto lógico. A tabela gerada pelo relacionamento N x N entre as entidades *A* e *B*, deve conter a chave primária da entidade *A* e a chave primária da entidade *B*.**

A Figura 22 mostra a tabela produzida como resultado do relacionamento N x N entre *Filme* e *Artista*. Em nosso projeto, chamamos essa tabela de *FilmeElenco*. É exatamente ela que registra o elenco dos filmes.

Vale lembrar que em nosso DER completo do BD filmes mostrado na Figura 2, existe outro relacionamento N x N, entre *Filme* e *Gênero* que também produz uma tabela no modelo lógico para armazenar os gêneros dos filmes (que poderia se chamar *FilmeGenero*). O DER da Figura 2, possui quatro entidades e dois relacionamentos N x N, resultando em um total de seis tabelas no modelo lógico.

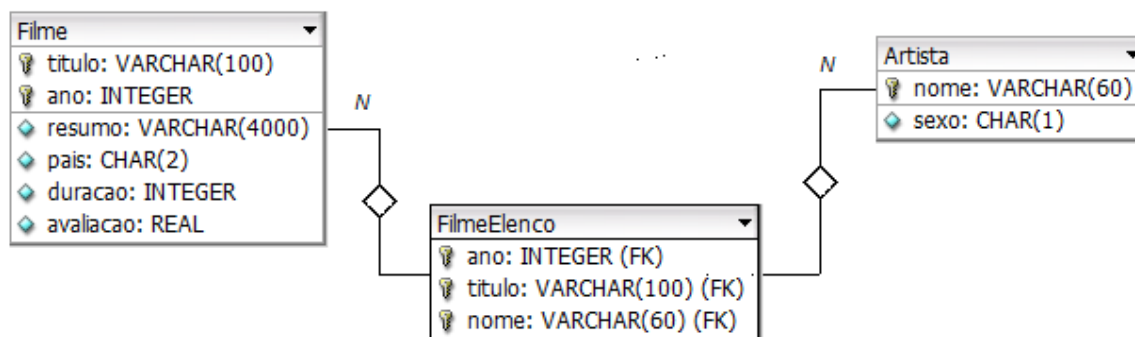
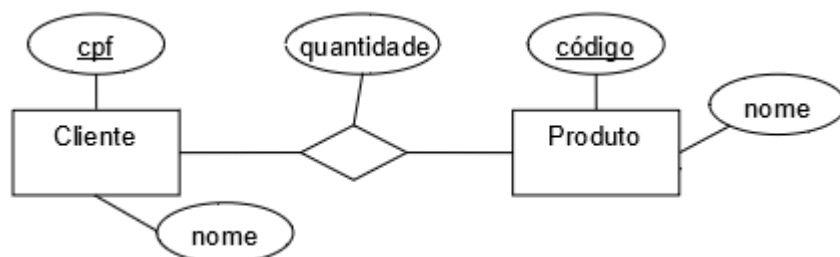


Figura 22. Regra 4 (exemplo no BD de filmes) - o relacionamento N x N entre *Filme* e *Artista* produz uma nova tabela, chamada *FilmeElenco*

Relacionamentos N x N Podem Possuir Atributos

Relacionamentos N x N são muito especiais já que podem possuir atributos. Para ver isso na prática, veja a figura abaixo onde mostramos o relacionamento “*Cliente compra Produto*” em um diagrama E-R. Trata-se de um relacionamento N x N, pois um cliente compra muitos produtos e um produto pode ser comprado por vários clientes. Neste relacionamento, o atributo “quantidade” pertence ao relacionamento compra.



Outras Regras – Mapeamentos Especiais

As Regras 1 a 4 tratam dos casos de mapeamento ER-Relacional mais comumente empregadas na prática. No entanto, relacionamentos especiais³ como auto-relacionamentos e generalizações também podem estar presentes em determinados modelos conceituais. Esta seção descreve a forma de mapear estes relacionamentos especiais.

Generalização e Especialização

A Figura 23 apresenta um exemplo de generalização / especialização em um DER, onde a entidade genérica *Produto* foi especializada em duas entidades, *Laptop* e *Impressora*.

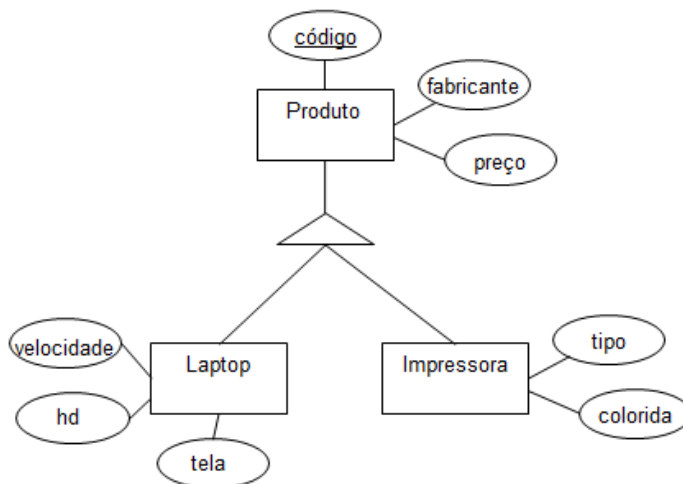


Figura 23. Generalização / especialização

³ A literatura utiliza a expressão ‘Modelo EER’ (*Enhanced Entity-Relationship Model*) para designar os modelos E/R que contém relacionamentos “especiais” ou “avançados”, tais como generalização / especialização.

Existem algumas técnicas distintas de realizar o mapeamento, sendo mais comum a utilização da seguinte abordagem em dois passos: (i) criar uma tabela distinta para cada entidade (respeitando assim a regra 1); (ii) implementar um relacionamento do tipo 1 x 1 entre cada entidade especializada e a entidade genérica. A PK da entidade genérica deverá então ser transportada como FK para cada entidade especializada. A Figura 24 mostra o resultado obtido por esta técnica no mapeamento da relação entre a entidade genérica *Produto* e as entidades especializadas *Laptop* e *Impressora*.



Figura 24. Mapeamento de relações do tipo generalização / especialização

Auto-Relacionamento

Uma entidade auto-relacionada é aquela cujos membros se relacionam com outros membros da própria unidade. Um exemplo deste tipo de relacionamento é apresentado abaixo:

- *Pessoa casa com Pessoa.*

Supondo que a entidade *Pessoa* seja definida por (cpf, nome, sexo, idade), a implementação deste auto-relacionamento pode ser realizada através da inclusão de um segundo campo de “cpf” na tabela *Pessoa*. Este campo será utilizado para armazenar o CPF do cônjuge de cada membro de *Pessoa* (Figura 25). Observe que “cpf_conjuge” é uma FK referenciando o campo “cpf” (PK) da própria tabela *Pessoa*!

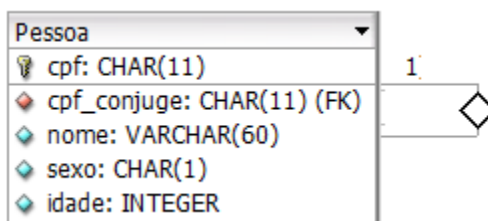


Figura 25. Mapeamento de auto-relacionamento

3.2 Chave Substituta

Uma chave substituta (*surrogate key*) é um campo que não tem significado para o minimundo (não representa uma propriedade real de uma entidade) e que é introduzida artificialmente em uma tabela do modelo lógico para servir como uma **chave primária**.

Mas por que isso é feito? Isso ocorre porque em muitas situações práticas, a chave primária “natural” de uma tabela é muito “complicada” ou consiste em um campo ou combinação de campos com tamanho muito grande.

Por exemplo: em nosso BD de filmes, a chave primária de *Filme* é o “título” (VARCHAR(100)) e o “ano” (INTEGER). Já em *Artista*, a chave primária é “nome” (VARCHAR(60)). Não é bom ter uma PK onde um dos campos é um VARCHAR(100) ou VARCHAR(60), pois uma PK pode sempre ser transportada como uma FK para várias outras tabelas (veja o exemplo do relacionamento entre *Filme* e *Artista* que foi apresentado na Figura 8), e isto acarretaria uma degradação da performance do BD, além de um aumento desnecessário no espaço ocupado.

Para resolver o problema, cria-se uma *surrogate key* chamada “CodFilme”, um simples número inteiro sequencial que passará a ser utilizado como PK de filme. A mesma coisa pode ser feita na tabela *Artista*. Assim, o BD se torna mais simples e compacto. Veja na Figura 26 como ficam as tabelas *Filme*, *Artista* e *FilmeElenco* com o uso de surrogate keys.

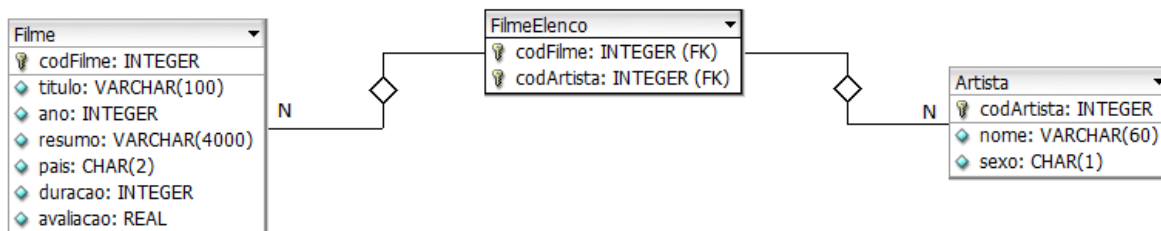


Figura 26. Tabelas com chaves substitutas

3.3 Notação Pé-de-Galinha em um Modelo Lógico

Conforme apresentado anteriormente, muitos projetistas preferem utilizar a Notação Pé-de-Galinha. A Figura 27 apresenta o relacionamentos 1 x N entre *Filme* e *País*, agora expresso na notação Crow’s Foot. No modelo lógico, o símbolo do “pé-de-galinha” ao lado de *Filme* indica que um membro de *País* pode estar associado a um ou mais membros de *Filme*. Já o símbolo de “dois traços” ao lado *País*, indica que um membro de *Filme* pode estar associado a um e somente um membro de *País*.

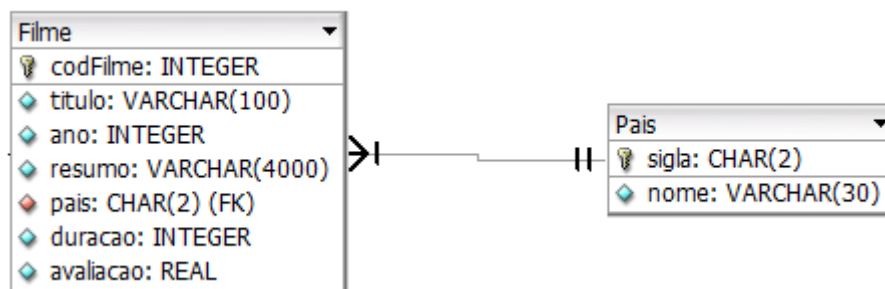


Figura 27. Representação do relacionamento “é produzido” entre as entidades *Filme* e *País* em um modelo lógico com o uso da notação Crow’s Foot.

3. Desnormalização

Considere a tabela *Cliente*(idCliente, nome, bairro, cidade, uf, cep), da qual uma possível instância é apresentada na Figura 28. Nesta tabela, "idCliente" é a PK.

idCliente	nome	bairro	cidade	uf	cep
1	Antônio	Jardim Oceânico	Rio de Janeiro	RJ	20230-010
2	Ana	Curicica	Rio de Janeiro	RJ	22780-080
3	Pedro	Icaraí	Niterói	RJ	24230-051
4	Carla	Praia do Canto	Vitória	ES	29050-320

Figura 28. Instância da tabela *Cliente*

Tendo em vista que existem nomes de cidades iguais em UFs diferentes e nomes de bairros iguais em cidades diferentes, as seguintes FD's são válidas em *Cliente*:

idCliente → {nome, bairro, cidade, uf, cep}
 cep → {cidade, uf}

Ok, pois "idCliente" é chave
PROBLEMA! violação da BCNF.

Sendo assim, de acordo com a teoria da normalização, seria necessário separar "cidade" e "uf" em uma nova tabela para evitar uma violação da BCNF (e da 3NF). Ou seja, caso desejássemos seguir a teoria da normalização com rigor, seria preciso dividir a tabela original. Ficaríamos com uma tabela *Cliente* com os atributos "idCliente", "nome", "bairro" e "cep" e uma nova tabela chamada *Cep* deveria ser criada com cada "cep" e suas respectivas "cidades" e "ufs".

No entanto, em situações como esta, muitas vezes os projetistas de BD optam por deliberadamente violar a 3FN ou a FNBC para deixar o projeto mais prático, mesmo que isto incorra na definição de uma tabela "desnormalizada".

No exemplo apresentado, a opção entre normalizar ou desnormalizar dependeria basicamente da importância do endereço (como um todo) para a aplicação. Caso o endereço esteja no sistema apenas com uma função cadastral, então provavelmente valeria a pena deixar a tabela da forma como ela está (ou seja, desnormalizada). Isto porque desnormalizando o BD, você diminui o número de tabelas e, por consequência, vai diminuir o número de SQLs que precisam de JOIN. O preço a pagar é redundância (risco de anomalias), mas em situações como a apresentada, isto vale a pena.

Por outro lado, se o endereço está envolvido em produção de estatísticas, pode ser interessante fazer uma normalização rigorosa, para garantir a consistência dos dados. Isso significaria ter uma tabela só de ufs, outra só de cidades, outra com os bairros de cada cidade (inclusive atribuindo um código único – surrogate key – para cada bairro), , outra com os CEPs etc. Dessa forma, seria possível, por exemplo, impedir o cadastro de diferenças na grafia dos bairros (o sistema não permitiria uma tupla com o valor "Jd.

Oceânico" e outra com o valor "JARDIM OCEÂNICO", já que todos os bairros estariam pré-cadastrados em uma tabela própria).

4. Um Exemplo Passo a Passo

Considere uma loja que vende madeiras e outros materiais para fabricantes de móveis. Cada venda da loja é atualmente registrada em uma nota de fornecimento em papel. As Figuras 29 e 30 representam dois exemplos relativos a vendas realizadas para dois clientes distintos.

Suponha que a loja deseje criar um BD para manter todas as informações referentes às notas de fornecimento de mercadorias. Na página a seguir, mostramos um exemplo de solução passo-a-passo que você poderá utilizar como modelo para resolver outros exercícios de projeto de BD.

Nota de Fornecimento de Mercadoria					
Data: 09/10/2017			Nº da Nota: 1019		
Dados do Cliente					
Id. Cliente	Nome	Telefone	Endereço		
31	Retrô Móveis	21-2142-0000	Av. Chile, 500		
Item	Cod. Produto	Produto	Quantidade	Preço (R\$)	Total
1	P87	Chapa de MDF	10	14,90	149,00
2	P23	Rodapé em Poliestierno	17	12,90	219,30
3	P44	Estrado	5	8,50	42,50

Figura 29. Nota de fornecimento de mercadoria (exemplo 1)

Nota de Fornecimento de Mercadoria					
Data: 11/10/2017			Nº da Nota: 1035		
Dados do Cliente					
Id. Cliente	Nome	Telefone	Endereço		
48	Móveis Alfa	21-3333-0000	Rua Frei Caneca, 1350		
Item	Cod. Produto	Produto	Quantidade	Preço (R\$)	Total
1	P20	Chapa de MDP	20	11,90	238,00
2	P87	Chapa de MDF	20	13,90	278,00

Figura 30. Nota de fornecimento de mercadoria (exemplo 2)

Passo 1: Resolveremos o problema **partindo da identificação de todas as entidades e relacionamentos**. Também é possível **identificar os atributos** de cada entidade (em problemas mais complexos isso pode ser mais difícil em um primeiro momento – nestes casos você irá ao menos tentar identificar os principais atributos das entidades).

Ao observar as Figuras 29 e 30, é possível concluir que:

- Uma *nota* é relativa a um único um cliente. Cada nota possui um número único e uma data de emissão.
- Uma nota contém diversos *itens*. Cada item representa um produto, comprado em uma determinada quantidade, por um determinado preço unitário.
- Um *cliente* pode estar associado a diversas notas (compras realizadas em dias diferentes). Cada cliente possui um código (id) único, nome, telefone e endereço.
- Cada *produto* tem um código e nome.

Passo 2: De acordo com as anotações feitas no passo 1, é possível elaborar o modelo conceitual para o BD, na forma de um DER. Este modelo é mostrado na Figura 31. Veja que o modelo possui três entidades (*Nota*, *Cliente* e *Produto*) e dois relacionamentos. Um dos relacionamentos é do tipo 1 x N (*Nota* e *Cliente*) e o outro do tipo N x N (*Nota* e *Produto*).

Neste momento, é importante registrar, mesmo que em um papel ou documento a parte, os atributos associados a cada entidade e relacionamento. Isto lhe ajudará a não ficar perdido no momento de realizar o projeto lógico.

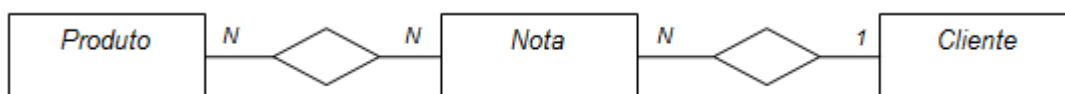


Figura 31. Modelo conceitual do BD da loja de madeiras

Passo 3: Elaborar o projeto lógico. Para tal é preciso:

- Aplicar as regras de mapeamento ER-Relacional sobre o modelo conceitual elaborado no Passo 2;
- Determinar os atributos, tipos e chaves de cada tabela, considerando o levantamento feito no Passo 1.

Como resultado, podemos chegar ao modelo lógico que é apresentado na Figura 32.

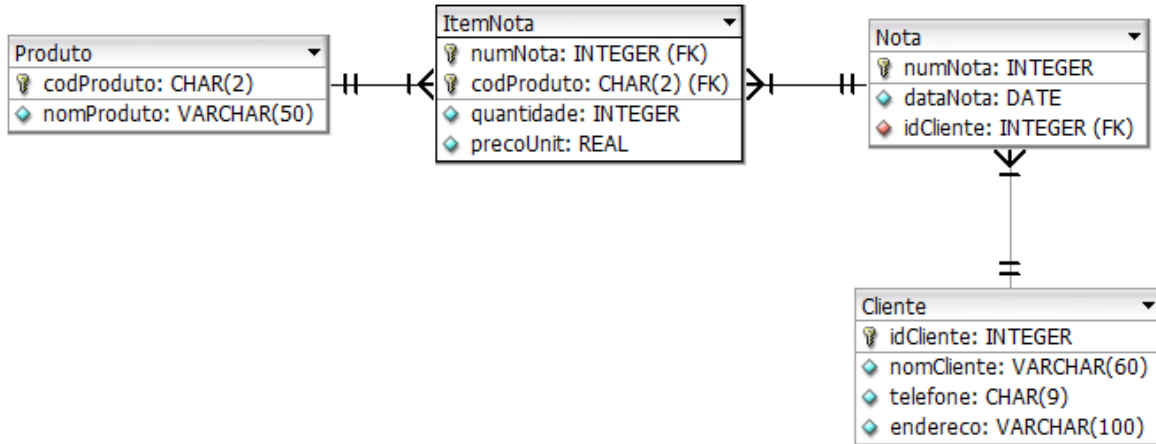


Figura 32. Projeto lógico do BD da loja de madeiras

COMENTÁRIOS:

- As três entidades do modelo conceitual (*Nota*, *Produto* e *Cliente*) se transformaram em três tabelas no modelo lógico (aplicação da regra de mapeamento 1).
- O campo “idCliente”, que é PK de *Cliente*, foi incluído como FK na tabela *Nota*, para possibilitar a implementação do relacionamento 1 x N entre *Cliente* e *Nota*.
- A tabela *ItemNota* “nasce” do relacionamento N x N entre *Produto* e *Nota*. A PK de *ItemNota* é {numNota, codProduto}, ou seja, a concatenação da PK de *Cliente* como a PK de *Produto*. Esta tabela possui ainda dois atributos próprios: “quantidade” e “precoUnit”.
- Para simplificar a resolução, não realizamos a normalização do endereço do cliente (por exemplo, separar em logradouro, bairro, cidade, UF e criar ao menos uma tabela separada para UF e outra para cidade). Ou seja, “endereço” foi definido como um único atributo do tipo VARCHAR(100) em *Cliente*. Mas na **maioria das situações práticas** é necessário realizar a separação, para que seja possível realizar consultas por cidade, bairro ou UF, por exemplo.
- Não é necessário armazenar o preço total no BD, pois este é um atributo derivado do preço unitário do produto e quantidade comprada (precoUnit x quantidade). Desta forma, seria redundante armazenar o preço total (acabariamos por dar margem à ocorrência de anomalias).
- Com relação ao preço, em muitos sistemas reais é necessário criar uma tabela auxiliar de histórico de preços para armazenar todo o histórico de preços de todos os produtos. Em nosso projeto utilizamos uma estratégia mais simples (que nem sempre pode ser utilizada em projetos reais): como temos o preço unitário na tabela *ItemNota*, podemos saber qual era o preço do produto na data em que aquela nota foi gerada.

- Também desconsideramos a informação da coluna “Item”, pois ela refere-se apenas à posição do item na nota de papel (se está na 1ª linha, 2ª linha, etc.).

5. Exercícios

Exercício 1. Um sistema que gerencia os **projetos de iniciação científica** de uma universidade possui as seguintes características:

- Cada projeto é identificado unicamente por um código, possui um título, ano, um orçamento previsto em R\$ e está associado a um único assunto (ex: Estatística, Física, Banco de Dados, etc.)
- Um assunto possui um nome e nenhum outro atributo.
- Um projeto possui um único pesquisador responsável e diversos alunos participantes.
- Pesquisadores possuem CPF, nome e nível (J=Júnior, P=pleno ou S=senior).
- Alunos possuem uma matrícula, nome, sexo e idade.

A partir do minimundo descrito, **faça o projeto lógico de um BD** para ser utilizado por este sistema.

Exercício 2. Considere um sistema gerador de *quizzes*, onde:

- Cada *quiz* é identificado unicamente por um código, possui uma data de geração e refere-se a um único tema (ex: Banco de Dados, Algoritmos, Big Data, etc.).
- Um tema possui um nome e nenhum outro atributo.
- Um *quiz* contém diversas questões.
- Cada questão está associada a um único tema.
- Cada questão possui um texto de até 400 caracteres representando o seu enunciado e possui diversas opções de respostas.
- Cada opção de resposta pertence a apenas uma questão, possui uma letra (A, B, C ou D), é descrita por um texto de até 255 caracteres e possui uma classificação (C=CORRETA ou I=INCORRETA).
- Uma questão pode aparecer em diferentes *quizzes*, sempre com as mesmas opções de respostas. Dentro de um *quiz*, a questão receberá um número que determinará a sua ordem (1, 2, ...).

A seguir, apresenta-se um exemplo de *quiz* sobre o tema “Banco de Dados” contendo apenas 2 questões. Neste *quiz*, a resposta correta da primeira questão é (B) e a da segunda é (A). Observe o exemplo e elabore os **projetos conceitual e lógico para o BD** deste sistema.

Quiz 1001 – Tema: *Banco de Dados* 20/06/2016

1. No Modelo Relacional uma tupla pode ser definida como:

- (A) Um índice.
- (B) Um conjunto de valores atômicos.
- (C) Uma coluna de uma relação.
- (D) Uma coleção de relações descrevendo um mini-mundo.

2. Qual dos seguintes itens melhor descreve a relação entre as linguagens R e SQL

- (A) Existem pacotes que possibilitam o uso de comandos SQL dentro de programas R.
- (B) R e SQL são as duas linguagens nativas dos SGBDs Relacionais
- (C) SQL é completamente incompatível com R.
- (D) R é uma linguagem NoSQL.

Exercício 3. Considere uma maternidade que deseja criar um banco de dados para registrar informações sobre os bebês nascidos, mães e equipes que realizam os partos. De acordo com os requisitos a seguir, **elabore os projetos conceitual e lógico para o BD:**

- Para cada parto ocorrido, deve-se armazenar a identificação do bebê, da mãe e da equipe médica. Considere que em cada parto, a equipe médica é sempre formada por apenas um médico e diversos enfermeiros. Deve ser ainda registrada a identificação do plano de saúde que cobriu os custos do parto, caso algum tenha sido utilizado.
- Para cada bebê nascido, as seguintes informações devem ser armazenadas: nome do bebê, data de nascimento, peso no nascimento e altura.
- Os seguintes dados sobre as mães devem ser registrados: RG (número da identidade), nome, endereço, telefone e data de nascimento.
- Um médico possui CRM (número único de inscrição no conselho de medicina), nome, e telefone celular.
- Um enfermeiro possui COREN (número único de inscrição no conselho de enfermagem) e nome.
- Para cada plano de saúde, deve-se armazenar o nome da empresa (ex: UNIPLAN) a categoria de plano (ex: ALFA, BETA, MASTER, etc.).

Considere que gêmeos representam dois partos diferentes. Além disso, não esqueça que uma mesma mãe pode realizar mais de um parto em datas diferentes